DAA ASSIGNMENT-7

M.NAVADEEP;

2024167344;

SUM OF SUBSETS:-

```c
#include <stdio.h>

int n, d, count = 0;
int w[20], x[20];

void sumOfSubsets(int s, int k, int r) {
    int i;
    x[k] = 1;

    if (s + w[k] == d) {
        count++;
        printf("Subset %d: { ", count);
        for (i = 0; i < n; i++) {
            if (x[i] == 1)
                printf("%d ", w[i]);
        }
        printf("}\n");
    } else if (s + w[k] + w[k + 1] <= d) {
        sumOfSubsets(s + w[k], k + 1, r - w[k]);
    }

    x[k] = 0;
    if ((s + r - w[k] >= d) && (s + w[k + 1] <= d)) {
        sumOfSubsets(s, k + 1, r - w[k]);
    }
}

int main() {
    int i, sum = 0;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements in ascending order:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &w[i]);
        sum += w[i];
    }
```

```
    printf("Enter the target sum: ");
    scanf("%d", &d);

    if (sum < d) {
        printf("No subset possible.\n");
    } else {
        printf("\nSubsets with sum %d:\n", d);
        sumOfSubsets(0, 0, sum);
        if (count == 0)
            printf("No subset found.\n");
    }

    return 0;
}
```

OUTPUT:-

Output Clear

```
Enter the number of elements: 5
Enter the elements in ascending order:
1 2 3 4 5
Enter the target sum: 5

Subsets with sum 5:
Subset 1: { 1 4 }
Subset 2: { 2 3 }
Subset 3: { 5 }


=== Code Execution Successful ===
```

N-queens problem:-

```
#include <stdio.h>
#include <math.h>
```

```c
int x[20];
int count = 0;

int canPlace(int k, int i) {
    int j;
    for (j = 1; j < k; j++) {
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void printBoard(int n) {
    int i, j;
    count++;
    printf("\nSolution %d:\n", count);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (x[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
    printf("Placement: ");
    for (i = 1; i <= n; i++)
        printf("%d ", x[i]);
    printf("\n");
}

void nQueens(int k, int n) {
    int i;
    for (i = 1; i <= n; i++) {
        if (canPlace(k, i)) {
            x[k] = i;
            if (k == n)
                printBoard(n);
            else
                nQueens(k + 1, n);
        }
    }
}

int main() {
    int n;
    printf("Enter the number of queens: ");
```

```
    scanf("%d", &n);

    printf("\nSolving %d-Queens Problem using Backtracking...\n", n);
    nQueens(1, n);

    if (count == 0)
        printf("No solution exists for %d queens.\n", n);
    else
        printf("\nTotal solutions: %d\n", count);

    return 0;
}
```

Output:-

Output Clear

```
Enter the number of queens: 4

Solving 4-Queens Problem using Backtracking...

Solution 1:
 .  Q  .  .
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .
Placement: 2 4 1 3

Solution 2:
 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .
Placement: 3 1 4 2

Total solutions: 2


=== Code Execution Successful ===
```

DFS:-

```
#include <stdio.h>
```

```c
int adj[20][20], visited[20], n;

void dfs(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v);

    for (i = 1; i <= n; i++) {
        if (adj[v][i] == 1 && visited[i] == 0)
            dfs(i);
    }
}

int main() {
    int edges, u, v, start, i;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Initialize adjacency matrix
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            adj[i][j] = 0;

    // Initialize visited array
    for (i = 1; i <= n; i++)
        visited[i] = 0;

    printf("Enter the edges (u v):\n");
    for (i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // Undirected graph
    }

    printf("Enter the starting vertex: ");
    scanf("%d", &start);

    printf("\nDFS Traversal starting from vertex %d: ", start);
    dfs(start);
    printf("\n");

    return 0;
}
```

OUTPUT:-

Output

Clear

```
Enter the number of vertices: 5
Enter the number of edges: 6
Enter the edges (u v):
0 1
0 2
1 3
1 4
2 4
3 4
Enter the starting vertex: 0

DFS Traversal starting from vertex 0: 0 1 3 4 2


=== Code Execution Successful ===
```

BFS:-

```c
#include <stdio.h>

int adj[20][20], visited[20], queue[20], n;

void bfs(int start) {
    int front = 0, rear = 0, v, i;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS Traversal starting from vertex %d: ", start);

    while (front < rear) {
        v = queue[front++];
        printf("%d ", v);

        for (i = 1; i <= n; i++) {
```

```c
            if (adj[v][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}

int main() {
    int edges, u, v, start, i, j;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Initialize adjacency matrix
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            adj[i][j] = 0;

    // Initialize visited array
    for (i = 1; i <= n; i++)
        visited[i] = 0;

    printf("Enter the edges (u v):\n");
    for (i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // Undirected graph
    }

    printf("Enter the starting vertex: ");
    scanf("%d", &start);

    printf("\n");
    bfs(start);

    return 0;
}
```

OUTPUT:-

Output

Clear

```
Enter the number of vertices: 5
Enter the number of edges: 6
Enter the edges (u v):
0 1
0 2
1 3
1 4
2 4
3 4
Enter the starting vertex: 0

BFS Traversal starting from vertex 0: 0 1 2 3 4


=== Code Execution Successful ===
```

Output

Clear

```
Enter the number of elements: 5
Enter the elements in ascending order:
1 2 3 4 5
Enter the target sum: 5

Subsets with sum 5:
Subset 1: { 1 4 }
Subset 2: { 2 3 }
Subset 3: { 5 }


=== Code Execution Successful ===
```